

Sức mạnh của JSF 2, Phần 1: Hợp lý hóa việc phát triển ứng dụng Web

Đơn giản hóa việc dẫn hướng, loại bỏ cấu hình XML và tiếp cận dễ dàng các tài nguyên bằng JSF 2

David Geary, Chủ tịch, Clarity Training, Inc.

Tóm tắt: Với phiên bản 2.0, Java™ Server Faces (JSF) giúp dễ dàng triển khai thực hiện các ứng dụng Web được Ajax hóa vững mạnh. Bài viết này khởi đầu một loạt bài ba phần của David Geary, một thành viên nhóm chuyên gia JSF 2.0, chỉ cho bạn cách tận dụng lợi thế của các tính năng mới trong JSF 2 như thế nào. Trong bài đăng này, bạn sẽ học cách sắp xếp hợp lý hóa việc phát triển bằng JSF 2 khi thay thế cấu hình XML bằng các chú giải và quy ước, đơn giản hoá việc dẫn hướng và truy cập dễ dàng vào các nguồn tài nguyên. Và bạn sẽ thấy cách sử dụng Groovy trong các ứng dụng JSF của bạn như thế nào.

Đang có một cuộc tranh luận về nơi tốt nhất để cho ra đời các khung công tác ứng dụng Web: các tháp ngà — ở đó các nhà trí thức hỏi đầu thảo luận — hay đối lại là thế giới thực, nơi mà các khung công tác được sinh ra từ những thử thách gắt gao của nhu cầu cấp bách. Về trực giác hình như là những thử thách thử thách gắt gao của nhu cầu cấp bách thắng thế hơn các nhà trí thức đầu hỏi và tôi cũng cho rằng trực giác sẽ đứng vững khi xem xét kỹ hơn.

JSF 1 đã được phát triển trong một tháp ngà và người ta cho rằng các kết quả không được ngoạn mục cho lắm. Nhưng JSF làm được một điều tốt — nó làm xuất hiện một thị trường với nhiều đối mới trong thế giới thực. Rất sớm, Facelets đã bắt đầu như là một sự thay thế mạnh mẽ cho JavaServer Pages (JSP). Sau đó, đã xuất hiện Rich Faces, một thư viện Ajax JSF thú vị; ICEFaces, một cách tiếp cận mới lạ cho Ajax với JSF; Seam; Spring Faces; các thành phần Woodstock; JSF Templating (tạo khuôn mẫu JSF); v.v. Tất cả các dự án JSF mã nguồn mở đó được xây dựng bởi các nhà phát triển, những người cần đến các chức năng mà họ đã triển khai thực hiện.

Nhóm chuyên gia JSF 2.0 (JSF 2.0 Expert Group) về cơ bản đã tiêu chuẩn hóa một số đặc tính tốt nhất từ những dự án mã nguồn mở đó. Mặc dù JSF 2 thực sự đã được một nhóm trí thức hỏi đầu vạch ra, nó cũng được dẫn dắt bởi các đối mới trong thế giới thực. Ngầm lại, công việc của nhóm chuyên gia tương đối dễ dàng vì chúng ta đã đang đứng trên vai của những người khổng lồ như Gavin King (Seam), Alexandr Smirnov (Rich Faces), Ted Goddard (ICEFaces) và Ken Paulson (JSF Templating). Thực vậy, tất cả những người khổng lồ đó đã ở trong nhóm chuyên gia JSF 2. Vì vậy trong nhiều khía cạnh, JSF 2 đã kết hợp các khía

cạnh tốt nhất của tháp ngà và thế giới thực. Và điều ấy được chứng tỏ. JSF 2 là một cải tiến hơn nhiều so với nguyên bản của nó.

Đây là bài đầu tiên trong một loạt bài ba phần có hai mục tiêu: chỉ ra cho bạn các đặc tính thú vị của JSF 2 và cho bạn thấy cách sử dụng tốt nhất các đặc tính đó, sao cho bạn có thể tận dụng lợi thế của những gì mà JSF 2 cung cấp. Tôi sẽ cắt ngang qua hai mối quan tâm đó bằng cách minh họa việc sử dụng JSF 2 với một số lời khuyên để sử dụng tốt nhất JSF. Dưới đây là những lời khuyên cho bài này:

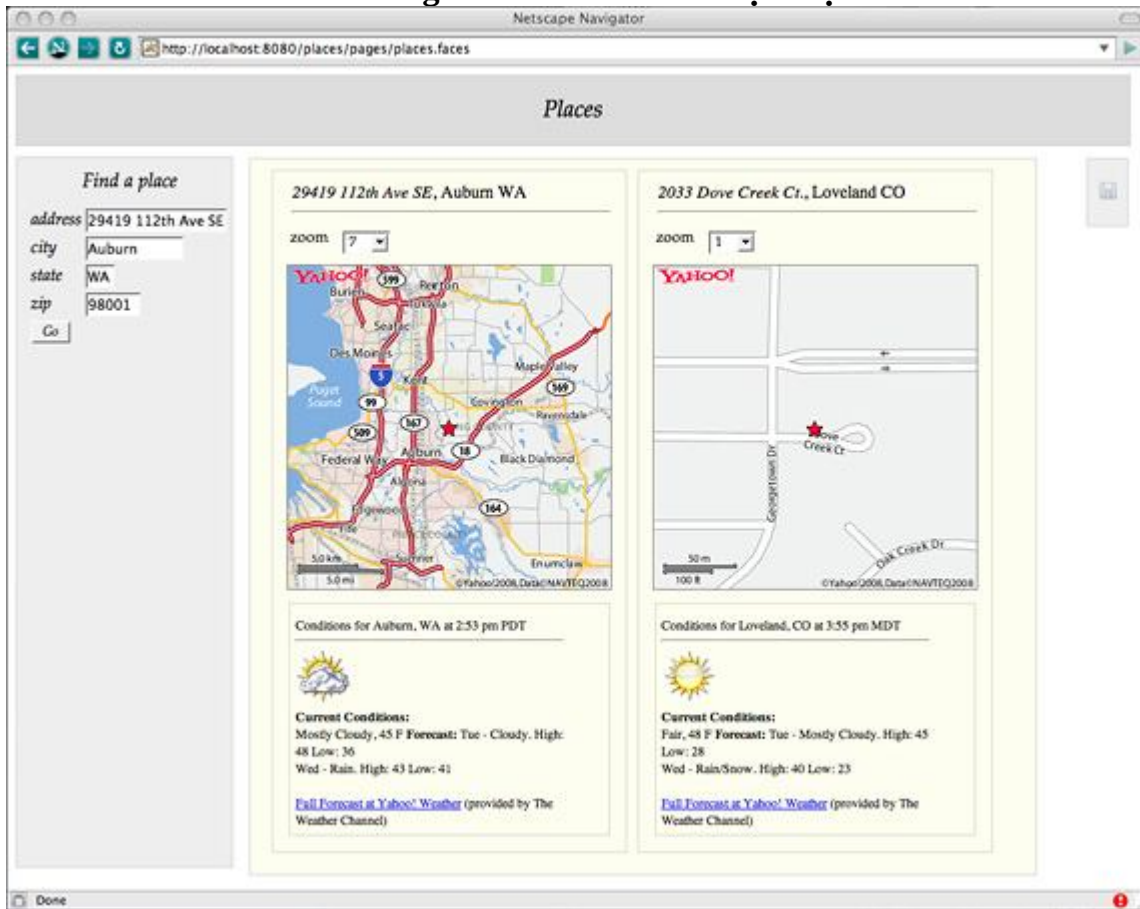
- Lời khuyên 1: Hãy từ bỏ cấu hình XML.
- Lời khuyên 2: Hãy đơn giản hóa việc dẫn hướng.
- Lời khuyên 3: Hãy sử dụng Groovy.
- Lời khuyên 4: Hãy tận dụng lợi thế trong xử lý tài nguyên,

Tuy nhiên, trước hết tôi sẽ giới thiệu ứng dụng ví dụ mà tôi sử dụng trong suốt loạt bài này. Mã nguồn ứng dụng cho bài này vẫn có sẵn để tải về.

Ví dụ trộn (mashup) các dịch vụ Web dựa trên bản đồ bắt buộc

Hình 1 cho thấy một ứng dụng trộn JSF — Tôi sẽ gọi nó là ứng dụng các địa điểm — có sử dụng các dịch vụ Web của Yahoo! để chuyển đổi các địa chỉ vào trong các bản đồ có các mức phóng to (zoom) và các dự báo thời tiết:

Hình 1. Xem bản đồ và thông tin thời tiết từ Các dịch vụ Web của Yahoo!

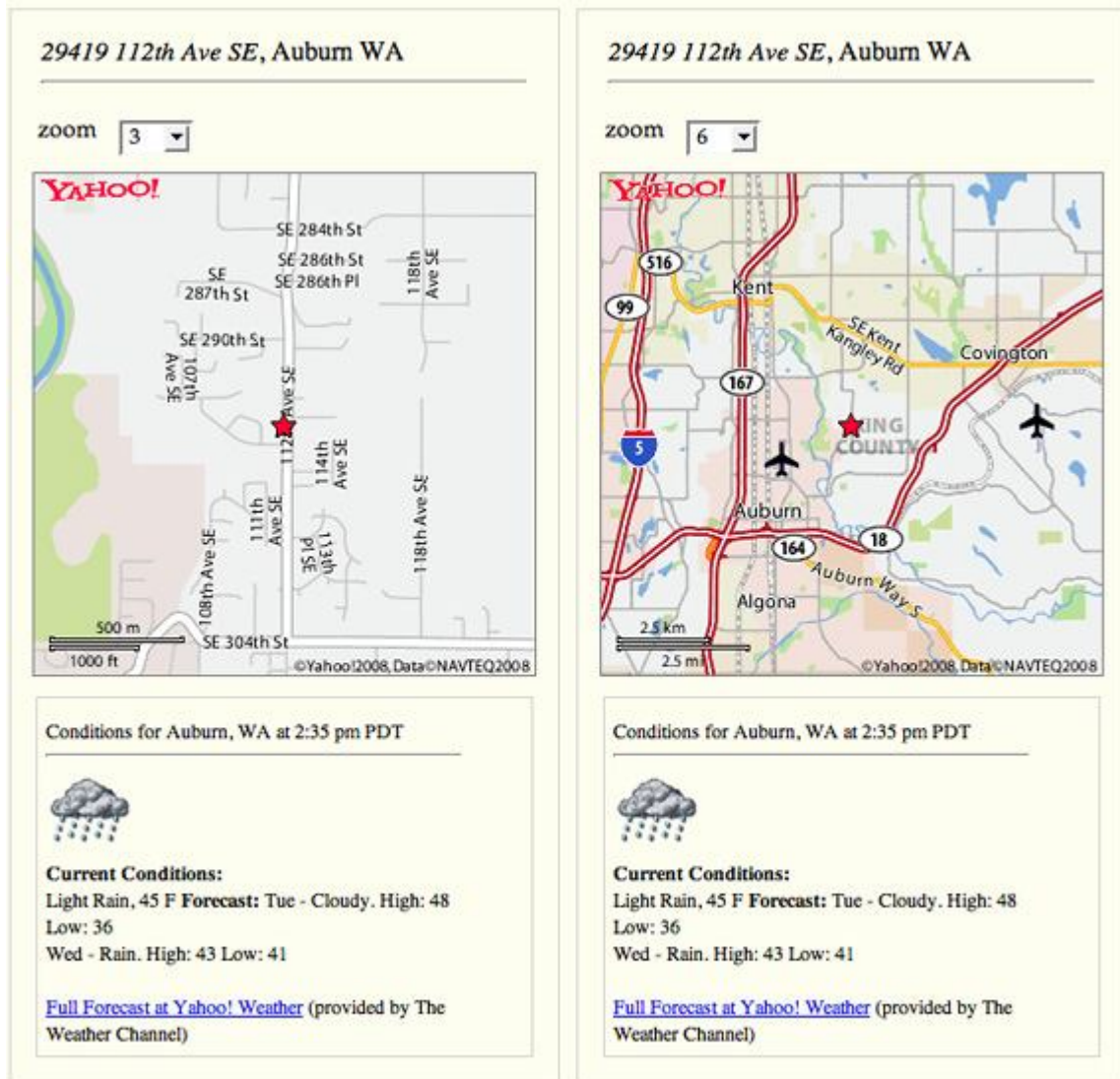


Để tạo một địa điểm, bạn điền vào biểu mẫu địa chỉ, kích hoạt nút Go và ứng dụng này chuyển địa chỉ đó đến hai dịch vụ Web: Yahoo! Maps (Các bản đồ của Yahoo!) và Yahoo! Weather (Thời tiết của Yahoo!)

Dịch vụ bản đồ này trả về 11 URL của bản đồ, trỏ đến các bản đồ của địa chỉ này, với các mức phóng to khác nhau, trên máy chủ của Yahoo!. Dịch vụ thời tiết chuyển trả lại một đoạn mã HTML được lắp ráp sẵn trước. Cả hai URL hình ảnh và các đoạn mã HTML dễ dàng được hiển thị trong một khung nhìn JSF, nhờ các thẻ `<h:graphicImage>` và `<h:outputText>`, tương ứng.

Ứng dụng các địa điểm cho phép bạn nhập vào bao nhiêu địa chỉ tùy thích. Bạn thậm chí có thể sử dụng cùng một địa chỉ nhiều hơn một lần, như chỉ ra trong Hình 2, thực sự muốn minh họa các mức phóng to:

Hình 2. Các mức phóng to



Ý chính của ứng dụng

Ứng dụng các địa điểm có bốn bean được quản lý, được liệt kê trong Bảng 1:

Bảng 1. Các bean được quản lý trong ứng dụng các địa điểm

Tên bean được quản lý

Lớp

Phạm vi

mapService	com.clarity.MapService	Ứng dụng
weatherService	com.clarity.WeatherService	Ứng dụng
places	com.clarity.Places	Phiên
place	com.clarity.Place	Yêu cầu

Chạy ứng dụng các địa điểm

Để chạy ứng dụng các địa điểm, bạn cần phải nhận được một mã nhận dạng (ID) của ứng dụng từ Yahoo! tại developer.yahoo.com/maps/ajax, để cho bạn có thể sử dụng Các dịch vụ Web của Yahoo! Nhấn vào nút **Get an App ID** (nhận một mã nhận dạng của ứng dụng) ở Dịch vụ Web các bản đồ của Yahoo! (Yahoo! Maps Web Service). Sau khi bạn có ID của mình, hãy thay thế YOUR_ID_HERE bằng ID của bạn trong MapService.java và WeatherService.java.

Ứng dụng lưu một danh sách về các Place, (địa điểm), được miêu tả trong Hình 1, trong phạm vi phiên làm việc và duy trì một Place trong phạm vi của yêu cầu (request). Ứng dụng này cũng cung cấp các API đơn giản trong các dịch vụ Web bản đồ và các dịch vụ Web thời tiết của Yahoo! với các bean được quản lý mapService và weatherService trong phạm vi ứng dụng, tương ứng.

Việc tạo các địa điểm rất đơn giản. Liệt kê 1 cho thấy mã cho biểu mẫu địa chỉ chứa trong khung nhìn của Hình 1:

Liệt kê 1. Các biểu mẫu địa chỉ

```
<h:form>
```

```
    <h:panelGrid columns="2">
```

```
        #{msgs.streetAddress} <h:inputText value="#{place.streetAddress}"
size="15"/>
```

```
#{msgs.city}      <h:inputText value="#{place.city}"      size="10"/>
```

```
#{msgs.state}     <h:inputText value="#{place.state}"      size="2"/>
```

```
#{msgs.zip}       <h:inputText value="#{place.zip}"      size="5"/>
```

```
<h:commandButton value="#{msgs.goButtonText}"
```

```
style="font-family:Palatino;font-style:italic"
```

```
action="#{place.fetch}"/>
```

```
</h:panelGrid>
```

```
</h:form>
```

Khi người dùng kích hoạt nút Go và gửi đi biểu mẫu, JSF gọi phương thức hành động của nút: `place.fetch()`. Phương thức này gửi thông tin từ các dịch vụ Web tới `Place.addPlace()`, nó tạo ra một cá thể `Place` (Địa điểm) mới, khởi trị cho cá thể bằng dữ liệu được chuyển tới phương thức đó và lưu trữ nó trong phạm vi của yêu cầu.

Liệt kê 2 hiển thị `Place.fetch()`:

Liệt kê 2. Phương thức `Place.fetch()`

```
public class Place {
```

```
...
```

```
private String[] mapUrls
```

```
private String weather
```

...

```
public String fetch() {
```

```
    FacesContext fc = FacesContext.getCurrentInstance()
```

```
    ELResolver elResolver = fc.getApplication().getELResolver()
```

```
    // Get maps
```

```
    MapService ms = elResolver.getValue(
```

```
        fc.getELContext(), null, "mapService")
```

```
    mapUrls = ms.getMap(streetAddress, city, state)
```

```
    // Get weather
```

```
    WeatherService ws = elResolver.getValue(
```

```
        fc.getELContext(), null, "weatherService")
```

```
    weather = ws.getWeatherForZip(zip, true)
```

```
    // Get places
```

```
    Places places = elResolver.getValue(
```

```

        fc.getELContext(), null, "places")

// Add new place to places

places.addPlace(streetAddress, city, state, mapUrls, weather)

return null

}

}

```

Place.fetch() sử dụng trình phân giải (resolver) biến của JSF để tìm các bean được quản lý mapService và weatherService và nó sử dụng các bean được quản lý đó để nhận được bản đồ và thông tin thời tiết từ Dịch vụ Web của Yahoo!. Sau đó fetch() gọi places.addPlace(), sử dụng bản đồ và thông tin thời tiết, cùng với địa chỉ, để tạo ra một Place mới trong phạm vi của yêu cầu.

Lưu ý là fetch() trả về null (không). Bởi vì fetch() là một phương thức hành động kết hợp với một nút nhấn, giá trị trả về null đó làm cho JSF nạp lại cùng khung nhìn, hiển thị tất cả các địa điểm trong phiên làm việc của người dùng, như chỉ ra trong Liệt kê 3:

Liệt kê 3. Hiển thị các địa điểm trong một khung nhìn

```

<ui:composition xmlns="http://www.w3.org/1999/xhtml"

    xmlns:f="http://java.sun.com/jsf/core"

    xmlns:h="http://java.sun.com/jsf/html"

    xmlns:ui="http://java.sun.com/jsf/facelets">

```



```
<h:form>
```

```
<!-- Iterate over the list of places -->
```

```
<ui:repeat value="#{places.placesList}" var="place">
```

```
<div class="placeHeading">
```

```
<h:panelGrid columns="1">
```

```
<!-- Address at the top -->
```

```
<h:panelGroup>
```

```
<div style="padding-left: 5px;">
```

```
<i><h:outputText value="#{place.streetAddress}"/></i>,</div>
```

```
<h:outputText value="#{place.city}"/>
```

```
<h:outputText value="#{place.state}"/>
```

```
<hr/>
```

```
</div>
```

```
</h:panelGroup>
```

```
<!-- zoom level prompt and drop down -->
```

```
<h:panelGrid columns="2">
```

```
<!-- prompt -->
```

```
<div style="padding-right: 10px;margin-bottom: 10px;font-size:14px">
```

```
#{msgs.zoomPrompt}
```

```
</div>
```

```
<!-- dropdown -->
```

```
<h:selectOneMenu onchange="submit()"
```

```
value="#{place.zoomIndex}"
```

```
valueChangeListener="#{place.zoomChanged}"
```

```
style="font-size:13px;font-family:Palatino">
```

```
<f:selectItems value="#{places.zoomLevelItems}"/>
```

```
</h:selectOneMenu>
```

```
</h:panelGrid>
```

```
<!-- The map -->
```

```
<h:graphicImage url="#{place.mapUrl}" style="border: thin solid gray"/>
```

```
</h:panelGrid>
```

```
<!-- The weather -->
```

```
<div class="placeMap">
```

```
<div style="margin-top: 10px;width:250px;">
```

```
<h:outputText style="font-size: 12px;"
    value="#{place.weather}"
    escape="false"/>

</div>

</div>

</div>

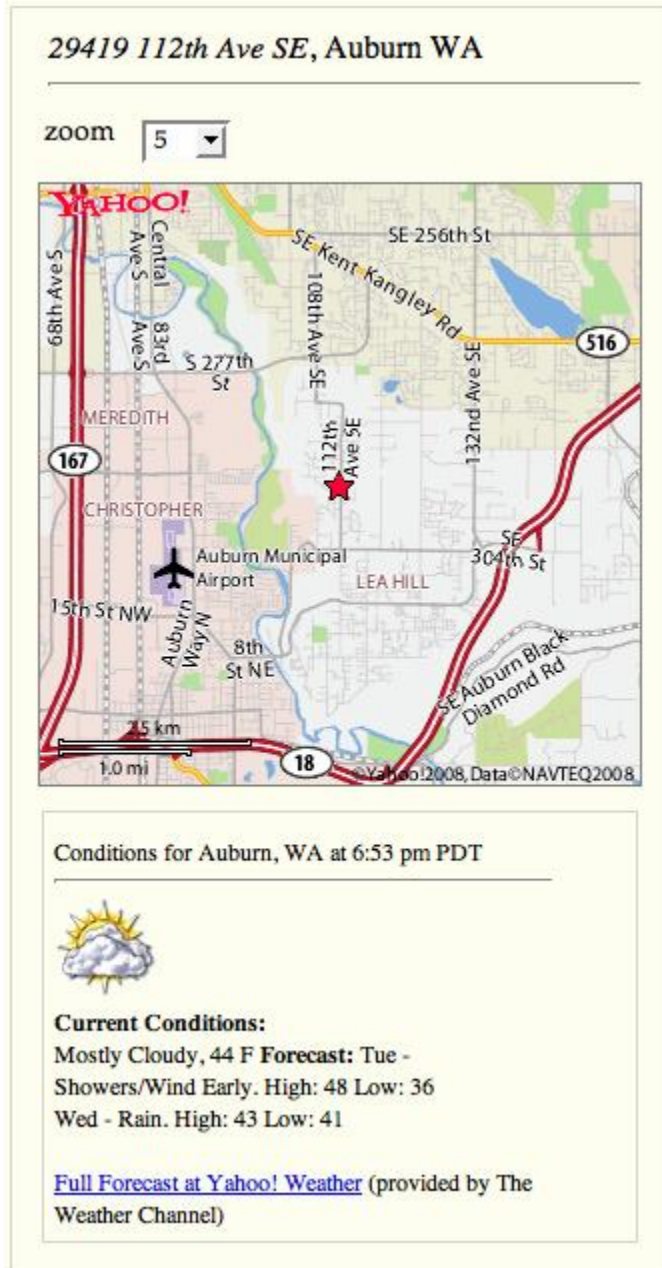
</ui:repeat>

</h:form>

</ui:composition>
```

Mã trong Liệt kê 3 sử dụng thẻ `<ui:repeat>` của Facelets để nối vòng qua danh sách các địa điểm được lưu trữ trong phiên làm việc của người dùng. Đối với mỗi địa điểm, kết quả đầu ra tương tự như Hình 3:

Hình 3. Một địa điểm được chỉ rõ trong một khung nhìn



Thay đổi mức phóng to

Trình đơn zoom (phóng to) (xem Hình 3 và Liệt kê 3) có một thuộc tính `onchange="submit()"`, do đó hàm JavaScript `submit()` gửi biểu mẫu bao quanh trình đơn khi người sử dụng chọn một mức phóng to. Kết quả của việc gửi biểu mẫu làm cho JSF gọi trình nghe thay đổi giá trị kết hợp với trình đơn — đó là phương thức `Place.zoomChanged()` được hiển thị trong Liệt kê 4:

Liệt kê 4. `Place.zoomChanged()`

```
public void zoomChanged(ValueChangeEvent e) {  
  
    String value = e.getComponent().getValue()  
  
    zoomIndex = (new Integer(value)).intValue()  
  
}
```

`Place.zoomChanged()` lưu trữ mức phóng to trong một biến thành phần của lớp `Place` có tên là `zoomIndex`. Vì sự dẫn hướng này không bị ảnh hưởng cho hành trình này đến máy chủ, nên JSF nạp lại cùng trang ấy và bản đồ được cập nhật với các mức phóng to mới, như sau: `<h:graphicImage url="#{place.mapUrl}..."/>`. Khi hình ảnh được lấy ra, JSF gọi `Place.getMapUrl()`, trả về URL của bản đồ cho mức phóng to hiện tại, như thể hiện trong Liệt kê 5:

Liệt kê 5. `Place.getMapUrl()`

```
public String getMapUrl() {  
  
    return mapUrls == null ? "" : mapUrls[zoomIndex]  
  
}
```

Một chút về Facelets

Nếu bạn đã sử dụng JSF 1, bạn có thể đã nhận thấy một số khác biệt tinh tế trong mã JSF 2 trong bài viết này. Đầu tiên, tôi đang sử dụng công nghệ hiển thị mới của JSF 2 — Facelets — thay vì JSP. Như bạn sẽ thấy trong các bài viết tiếp theo của loạt bài này, Facelets cung cấp nhiều tính năng mạnh mẽ để giúp bạn triển khai thực hiện các giao diện người dùng mạnh mẽ, linh hoạt và mở rộng được. Thế nhưng trong các liệt kê mã trước đây, tôi không đề cập nhiều đến khả năng này.

Tuy nhiên, một trong nhiều cải tiến nhỏ mà Facelets mang lại cho JSF là khả năng đặt các biểu thức giá trị của JSF trực tiếp vào trang XHTML của bạn; ví dụ, trong Liệt kê 1, tôi đã đặt các biểu thức như `{msgs.city}` trực tiếp vào trong trang. Với JSF 1, bạn phải bao bọc biểu thức đó trong một thẻ `<h:outputText>`, như thể này: `<h:outputText value="{msgs.city}"/>`. Tuy nhiên, cần hiểu rõ là bạn luôn phải áp mã thoát (escape) đối với bản đến từ đầu vào của người sử dụng vì lý do an ninh, do đó, ví dụ, trong Liệt kê 3 tôi sử dụng `<h:outputText>`, để hiển thị thông tin địa điểm, vì theo mặc định, thẻ này áp mã thoát cho văn bản của nó.

Một điều khác cần lưu ý, từ phối cảnh của Facelets, là thẻ `<ui:composition>` trong Liệt kê 3. Thẻ đó biểu thị rằng trang XHTML trong Liệt kê 3 dự tính là sẽ được chứa trong các trang XHTML khác. Các cấu kiện (compositions) của Facelets là thành phần trung tâm của việc *tạo khuôn mẫu* (templating) Facelets, nó tương tự với khung công tác Tiles rất phổ biến của Apache. Trong một bài viết tiếp theo của loạt bài này, tôi sẽ thảo luận về tạo khuôn mẫu Facelets và cho bạn thấy làm thế nào để cấu trúc các khung nhìn của bạn theo mẫu *phương thức kết hợp* (Composed Method) của Smalltalk.

Cho đến nay, ngoài việc sử dụng Facelets, đoạn mã trên không khác về cơ bản với JSF 1. Bây giờ tôi sẽ cho bạn thấy một số khác biệt đáng kể hơn. Khác biệt lớn thứ nhất là số lượng cấu hình XML mà bạn sẽ viết cho các ứng dụng JSF 2.

Lời khuyên 1: Hãy từ bỏ cấu hình XML

Cấu hình XML cho các ứng dụng Web luôn luôn đáng ngờ — nó tẻ nhạt và dễ bị lỗi và tốt nhất nên được giao cho một khung công tác, có thể là thông qua các chú giải, các quy ước hoặc các ngôn ngữ đặc thù riêng cho lĩnh vực ứng dụng. Với vai trò các nhà phát triển, chúng ta sẽ có thể tập trung vào việc giải quyết vấn đề, thay vì phải xoay xở với nhiều chi tiết vụn vặt trong tài liệu XML dài dòng.

Như một ví dụ minh họa tốt, Liệt kê 6 cho thấy 20 dòng XML cần thiết để khai báo các bean được quản lý trong ứng dụng địa điểm với JSF 1:

Liệt kê 6. Các khai báo bean-quản lý cho JSF 1

```
<managed-bean>
```

```
<managed-bean-class>com.clarity.MapService</managed-bean-class>  
<managed-bean-name>mapService</managed-bean-name>  
<managed-bean-scope>application</managed-bean-scope>  
</managed-bean>
```

```
<managed-bean>  
  <managed-bean-class>com.clarity.WeatherService</managed-bean-class>  
  <managed-bean-name>weatherService</managed-bean-name>  
  <managed-bean-scope>application</managed-bean-scope>  
</managed-bean>
```

```
<managed-bean>  
  <managed-bean-class>com.clarity.Places</managed-bean-class>  
  <managed-bean-name>places</managed-bean-name>  
  <managed-bean-scope>session</managed-bean-scope>  
</managed-bean>
```

```
<managed-bean>  
  <managed-bean-class>com.clarity.Place</managed-bean-class>  
  <managed-bean-name>place</managed-bean-name>  
  <managed-bean-scope>request</managed-bean-scope>  
</managed-bean>
```

Với JSF 2, XML biến mất và thay vào đó bạn chú giải các lớp của mình, như chỉ ra trong Liệt kê 7:

Liệt kê 7. Các chú giải bean-quản lý cho JSF 2

```
@ManagedBean(eager=true)
```

```
public class MapService {
```

```
    ...
```

```
}
```

```
@ManagedBean(eager=true)
```

```
public class WeatherService {
```

```
    ...
```

```
}
```

```
@ManagedBean()
```

```
@SessionScoped
```

```
public class Places {
```

```
    ...
```

```
}
```

```
@ManagedBean()
```



```
@RequestScoped

public class Place {

    ...

}
```

Theo quy ước, tên của một bean được quản lý giống như tên lớp, với chữ cái đầu của tên lớp được chuyển đổi từ chữ hoa thành chữ thường. Vì vậy, các bean được quản lý được tạo ra từ Liệt kê 7, chẳng hạn, từ trên xuống dưới, là: mapService, weatherService, places và place. Bạn cũng có thể chỉ định rõ ràng một tên bean được quản lý bằng thuộc tính name của chú giải ManagedBean, như thế này: @ManagedBean(name = "place").

Trong Liệt kê 7, tôi sử dụng thuộc tính eager (háo hức) cho các bean được quản lý mapService và webService. Khi thuộc tính eager là true (đúng), JSF tạo ra bean được quản lý lúc khởi động và đặt nó trong phạm vi ứng dụng.

Bạn cũng có thể thiết lập các thuộc tính bean được quản lý với chú giải @ManagedProperty. Bảng 2 cho thấy danh sách đầy đủ của các chú giải bean được quản lý của JSF 2:

Bảng 2. Các chú giải bean được quản lý của JSF 2 (các chú giải @...Scoped chỉ hợp lệ với @ManagedBean)

Chú giải bean được quản lý	Mô tả	Các thuộc tính
@ManagedBean	Đăng ký một cá thể của lớp này như là một bean được quản lý và đặt nó trong phạm vi được chỉ rõ bằng một trong các chú giải @...Scoped. Nếu không eager, chỉ rõ phạm vi nào, JSF đặt bean trong phạm vi của yêu cầu và nếu không chỉ rõ tên là gì, JSF chuyển đổi chữ cái đầu tiên trong className (tên lớp) thành chữ thường để có một tên bean được quản lý; ví dụ,	name

nếu className là UserBean, thì JSF tạo ra một bean được quản lý có tên là userBean. Cả hai thuộc tính eager và name là tùy chọn.

Chú giải này phải được sử dụng với một lớp Java triển khai thực hiện một hàm tạo không có đối số.

Thiết lập một thuộc tính của một bean được quản lý. Chú giải này phải được đặt trước khai báo biến thành phần của lớp. Thuộc tính name xác định tên của thuộc tính, mà mặc định là tên của biến thành phần. Thuộc tính value là giá trị của thuộc tính và có thể hoặc là một chuỗi ký tự hoặc là một biểu thức JSF, ví dụ như #{...}. value, name

@ManagedProperty

@ApplicationScoped Lưu trữ bean được quản lý trong phạm vi ứng dụng.

@SessionScoped Lưu trữ bean được quản lý trong phạm vi phiên làm việc.

@RequestScoped Lưu trữ bean được quản lý trong phạm vi của yêu cầu.

@ViewScoped Lưu trữ bean được quản lý trong phạm vi khung nhìn.

@NoneScoped Xác định rằng bean được quản lý không có phạm vi nào. Các bean được quản lý không có phạm vi nào rất hữu ích khi chúng được tham chiếu bởi các bean khác.

@CustomScoped Lưu trữ bean được quản lý trong một phạm vi tùy chỉnh. value

Một phạm vi tùy chỉnh đơn giản chỉ là một bản đồ mà các tác giả của trang có thể truy cập vào. Bạn có

thể kiểm soát bằng lập trình tầm nhìn và vòng đời của bean trong các phạm vi tùy chỉnh. Thuộc tính value trở tới một bản đồ.

Loại bỏ các khai báo bean được quản lý khỏi faces-config.xml làm giảm đáng kể XML của bạn, nhưng bạn có thể từ bỏ hầu như tất cả những cái đó với JSF 2 thông qua hoặc các chú giải, như tôi đang làm với các bean được quản lý, hoặc các quy ước, như đã làm cho việc xử lý dẫn hướng đơn giản hóa của JSF 2.

Lời khuyên 2: Đơn giản hóa dẫn hướng

Trong JSF 1, việc dẫn hướng đã được xác định trong XML. Ví dụ, để đi từ login.xhtml đến places.xhtml, bạn có thể sử dụng quy tắc dẫn hướng trong Liệt kê 8:

Liệt kê 8. Các quy tắc cấu hình dẫn hướng và các trường hợp đối với JSF 1

```
<navigation-rule>

  <navigation-case>

    <from-view-id>/pages/login.xhtml</from-view-id>

    <outcome>places</outcome>

    <to-view-id>/pages/places.xhtml</to-view-id>

  </navigation-case>

</navigation-rule>
```

Để thoát khỏi XML trong Liệt kê 8, bạn có thể tận dụng lợi thế của quy ước dẫn hướng của JSF 2: JSF thêm .xhtml vào cuối hành động của nút nhấn và nạp tệp đó. Điều đó có nghĩa là bạn không cần các chú giải hay bất cứ điều gì khác hơn là các

quy ước để thoát khỏi hoàn toàn việc viết quy tắc dẫn hướng. Trong Liệt kê 9, hành động của nút này là places, do đó, JSF nạp places.xhtml:

Liệt kê 9. Dẫn hướng theo quy ước

```
<h:commandButton id="loginButton"
value="#{msgs.loginButtonText}"
action="places"/>
```

Với Liệt kê 9 không có yêu cầu XML dẫn hướng nào. Nút trong Liệt kê 9 nạp places.xhtml, nhưng chỉ khi tệp đó ở trong cùng thư mục với tệp có chứa nút đó. Nếu hành động không bắt đầu bằng một dấu gạch chéo (/), JSF giả định rằng đó là một đường dẫn tương đối. Nếu bạn muốn rõ ràng hơn, bạn có thể chỉ rõ một đường dẫn tuyệt đối, như thể hiện trong Liệt kê 10:

Liệt kê 10. Dẫn hướng bằng các đường dẫn tuyệt đối

```
<h:commandButton id="loginButton"
value="#{msgs.loginButtonText}"
action="/pages/places"/>
```

Khi người dùng kích hoạt nút trong Liệt kê 10, JSF nạp tệp /pages/places.xhtml.

Theo mặc định, JSF chuyển tiếp từ trang XHTML sang một trang khác, nhưng thay vào đó bạn có thể chuyển hướng bằng cách chỉ rõ tham số faces-redirect, như minh họa trong Liệt kê 11:

Liệt kê 11. Dẫn hướng bằng chuyển hướng

```
<h:commandButton id="loginButton"
value="#{msgs.loginButtonText}"
action="places?faces-redirect=true"/>
```

Lời khuyên 3: Sử dụng Groovy

Điều tốt nhất về công nghệ Java không phải là ngôn ngữ Java, mà là máy ảo Java (JVM). Các ngôn ngữ mạnh mẽ, mới và sáng tạo như Scala, JRuby và Groovy chạy trên JVM, cho bạn những không gian khác để viết mã trong đó. Groovy — tên không hay nhưng là một pha trộn có thể làm nhiều việc của Ruby, Smalltalk và ngôn ngữ Java — là một trong những ngôn ngữ được ưa chuộng nhất trong các ngôn ngữ ấy (xem Tài nguyên).

Có nhiều lý do để sử dụng Groovy, bắt đầu từ thực tế là nó ngắn gọn hơn nhiều và mạnh hơn nhiều so với người anh em họ thứ hai của nó, đó là ngôn ngữ Java. Thêm hai lý do: không có các dấu chấm phẩy và không yêu cầu phải ép kiểu (casting).

Bạn có thể đã không nhận thấy, nhưng Liệt kê 2, dành cho lớp Place, được viết bằng Groovy. Việc thiếu các dấu chấm phẩy là một manh mối, nhưng cũng hãy lưu ý dòng mã này: `MapService ms = elResolver.getValue(...)`. Với mã Java, tôi sẽ phải ép kiểu kết quả của `ElResolver.getValue()`, vì phương thức đó trả về kiểu `Object` (Đối tượng). Groovy thực hiện việc ép kiểu thay cho tôi.

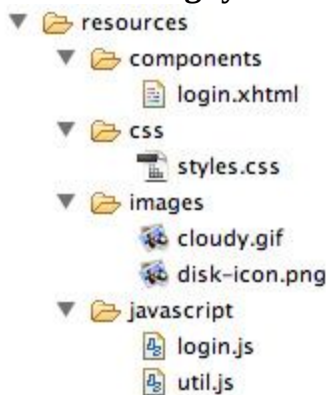
Bạn có thể sử dụng Groovy cho bất kỳ các tạo phẩm JSF nào mà bạn thường có thể viết bằng mã Java — ví dụ, các thành phần, các trình biểu hiện (renderer), các trình xác nhận hợp lệ (Validator) và các trình chuyển đổi (converter). Trong thực tế, điều này không có gì mới với JSF 2 — vì các tệp nguồn Groovy biên dịch thành bytecode của Java, bạn có thể sử dụng các tệp `.class` được Groovy sinh ra như thể chúng được `javac` tạo ra. Tất nhiên, ngay khi bạn đã thực hiện được việc này, bạn sẽ muốn biết làm thế nào để triển khai-nóng mã nguồn Groovy và đối với người sử dụng Eclipse, câu trả lời thật đơn giản: hãy tải về và cài đặt trình cắm thêm (plug-in) Groovy Eclipse (xem Tài nguyên). Mojarra, một triển khai thực

hiện JSF của Sun, đã hỗ trợ rõ ràng cho Groovy kể từ phiên bản 1.2_09 (xem Tài nguyên).

Lời khuyên 4: Sử dụng các trình xử lý tài nguyên

JSF 2 cung cấp một cơ chế tiêu chuẩn để định nghĩa và truy cập tài nguyên. Bạn đặt các tài nguyên của bạn trong một thư mục mức cao nhất có tên là `resources` (tài nguyên) và sử dụng một số thẻ JSF 2 để truy cập các tài nguyên này trong các khung nhìn của bạn. Ví dụ, Hình 4 cho thấy tài nguyên dành cho ứng dụng các địa điểm:

Hình 4. Tài nguyên của ứng dụng các địa điểm



Yêu cầu duy nhất cho một tài nguyên là nó nằm trong thư mục `resources` hoặc một thư mục con từ đó. Bạn có thể đặt tên các thư mục con của thư mục tài nguyên bất cứ là gì bạn muốn.

Trong mã khung nhìn của bạn, bạn có thể truy cập tài nguyên bằng một vài thẻ JSF 2: `<h:outputScript>` và `<h:outputStylesheet>`. Các thẻ đó làm việc phối hợp với thẻ `<h:head>` và `<h:body>` của JSF 2, như thể hiện trong Liệt kê 12:

Liệt kê 12. Truy cập tài nguyên trong XHTML

```
<html xmlns="http://www.w3.org/1999/xhtml">
```

```
xmlns:ui="http://java.sun.com/jsf/facelets"
```

```
xmlns:h="http://java.sun.com/jsf/html">
```

```
<h:head>
```

```
...
```

```
</h:head>
```

```
<h:body>
```

```
<h:outputStylesheet library="css" name="styles.css" target="body"/>
```

```
<h:outputScript library="javascript" name="util.js" target="head"/>
```

```
...
```

```
</h:body>
```

```
</html>
```

Các thẻ `<h:outputScript>` và `<h:outputStylesheet>` có hai thuộc tính để nhận biết kịch bản lệnh hay bảng định kiểu (stylesheet), tương ứng là: `library` và `name`. Tên `library` tương ứng với thư mục, trong thư mục `resources`, nơi tài nguyên được lưu trữ. Ví dụ, nếu bạn có một bảng định kiểu trong một thư mục `resources/css/en`, `library` sẽ là `css/en`. Thuộc tính `name` là tên của chính tài nguyên đó.

Tài nguyên có thể định vị lại được

Điều quan trọng là các nhà phát triển có thể xác định họ muốn tài nguyên của họ xuất hiện ở đâu trong một trang. Ví dụ, nếu bạn đặt mã JavaScript trong phần thân của một trang, trình duyệt sẽ thi hành mã JavaScript khi nạp trang. Mặt khác, nếu bạn đặt mã JavaScript trong phần đầu của một trang, mã JavaScript đó sẽ chỉ được thi hành khi được gọi. Vì vị trí của tài nguyên có thể ảnh hưởng đến cách nó được sử dụng, nên bạn cần có khả năng xác định bạn muốn tài nguyên kết thúc ở đâu.

Các tài nguyên JSF 2 *có thể định vị lại*, nghĩa là bạn có thể xác định vị trí trong trang mà bạn muốn đặt chúng. Bạn chỉ rõ vị trí đó bằng thuộc tính target (đích); ví dụ, trong Liệt kê 12, tôi đặt CSS trong phần thân và mã JavaScript trong phần đầu.

Đôi khi bạn cần truy cập vào một tài nguyên bằng cách sử dụng ngôn ngữ biểu thức (EL) của JSF. Ví dụ, Liệt kê 13 cho thấy làm thế nào bạn có thể truy cập một hình ảnh bằng <h:graphicImage>:

Liệt kê 13. Truy cập vào tài nguyên bằng ngôn ngữ biểu thức của JSF

```
<h:graphicImage value="#{resource['images:cloudy.gif']}"/>
```

Không dùng ngôn ngữ biểu thức cho Liệt kê 13

Phải thừa nhận rằng, cú pháp trong Liệt kê 13 là hơi khác thường. Thực tế nó truy cập một bản đồ được JSF tạo ra để lưu trữ tài nguyên, vì vậy bạn hiếm khi cần sử dụng cú pháp đó. Thực vậy, bạn có thể truy cập các hình ảnh bằng <h:graphicImage/> không cần dùng EL, như thế này: <h:graphicImage library="images" name="cloudy.gif"/>

Cú pháp để truy cập vào tài nguyên trong một biểu thức EL là resource['*LIBRARY:NAME*'], ở đây *LIBRARY* và *NAME* tương ứng với các thuộc tính library và name của các thẻ <h:outputScript> và <h:outputStylesheet>.

Vẫn còn tiếp tục

Cho đến nay tôi chỉ mới bàn sơ qua mặt ngoài của các đặc tính JSF 2, đó là các chú giải của bean được quản lý, đơn giản hóa việc dẫn hướng và hỗ trợ cho tài nguyên. Trong hai bài viết còn lại trong loạt bài này, tôi sẽ khảo sát tỉ mỉ Facelets, các thành phần phức hợp của JSF 2 và sự hỗ trợ làm sẵn cho Ajax.

Mục lục

- Ví dụ trộn (mashup) các dịch vụ Web dựa trên bản đồ bắt buộc
- Lời khuyên 1: Hãy từ bỏ cấu hình XML
- Lời khuyên 2: Đơn giản hóa dẫn hướng
- Lời khuyên 3: Sử dụng Groovy
- Lời khuyên 4: Sử dụng các trình xử lý tài nguyên
- Vẫn còn tiếp tục